1.

a. Let's take p=7 and q=13

n = p x q
= 7 x 13
= 91

ϕ(n) = (p – 1) (q – 1)
= (7 – 1) (13 – 1)
= 6 x 12
= 72

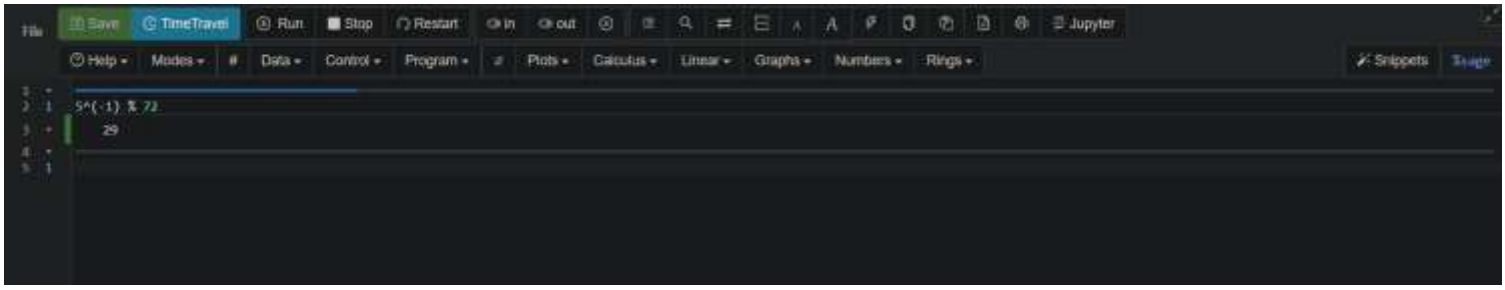
Let's take e = 5 since 1 < e < ϕ(n)

D is calculated where (e x d mod ϕ(n)) = 1

D = e^-1 mod ϕ(n)
= 5^-1 mod 72
= 29

Public key (n, e)
(91, 5) - sender

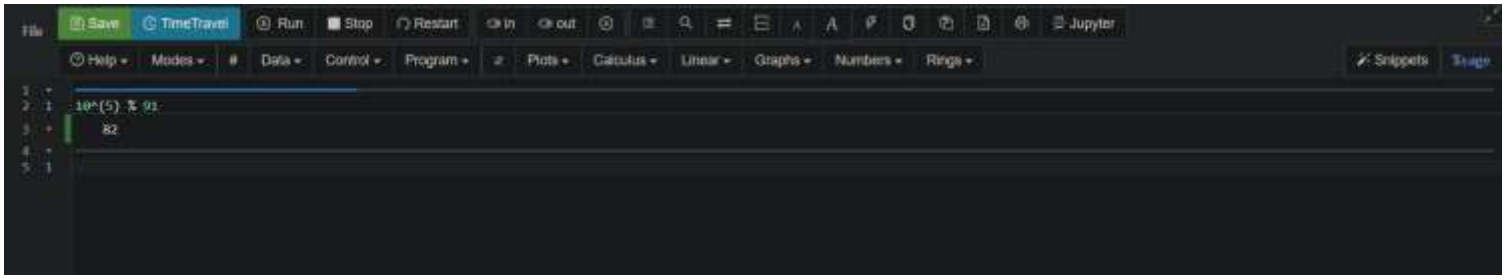
Private key (n, d)
(91, 29) - private



b. Let m = 10

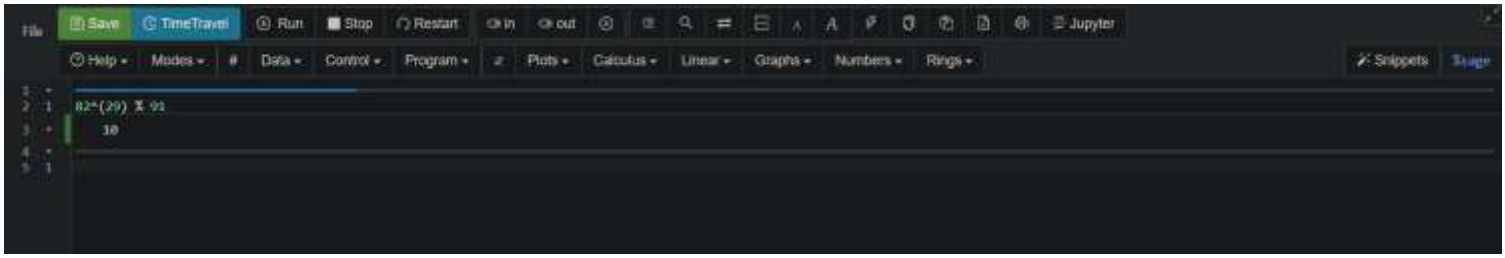
**Encryption**

C = P^e mod n
= 10^5 mod 91
82



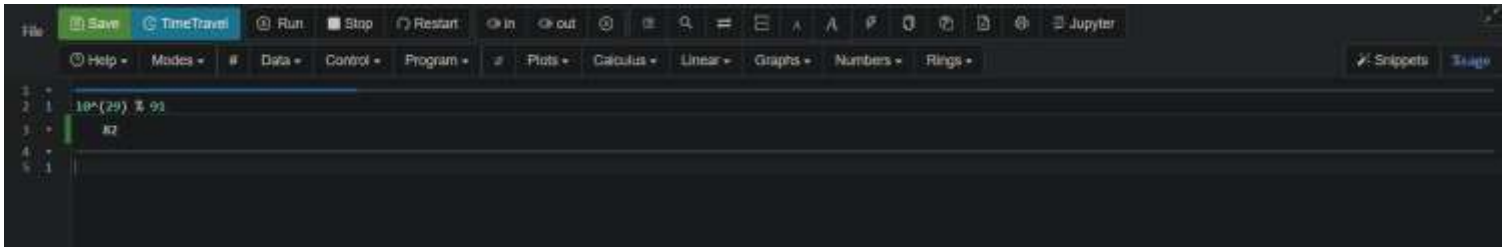

**Decryption**

P = C^d mod n
= 82^29 mod 91
= 10



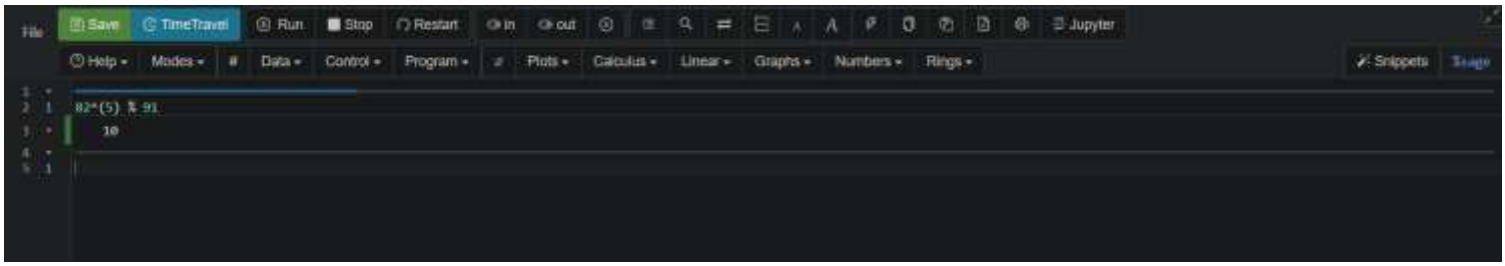
c. Sender A signs M using its receivers B private key, d.

Sign = m^d mod n
= 10^29 mod 91
= 82




B verifies the signature using A's public key, e.

m' = S^e mod n
= 82^5 mod 91
= 10




If m = m', then the signature is verified.

d. A encrypts the message m and then signs it. B verifies the signature and decrypts the message.

   This ensures confidentiality, Authentication, and Non-repudiation.

   - Confidentiality – Only B can decrypt the message because only B has the private key to decrypt.
   - Authentication – B can verify that the message came from A by using A's public key to verify the signature.
   - Non-repudiation – A cannot deny sending the message since only A's private key could have produced the signature that B successfully verified.

e. RSA's security relies on the difficulty of factoring large composite numbers. Let's say we have a composite number n which is a product of two large prime numbers, p and q. If we can efficiently factor n, we can find p and q and break RSA encryption.

   Suppose we have an RSA public key consisting of (**n**) and (**e**), where (**n**) is the product of two prime numbers (**p**) and (**q**), and (**e**) is the public exponent.

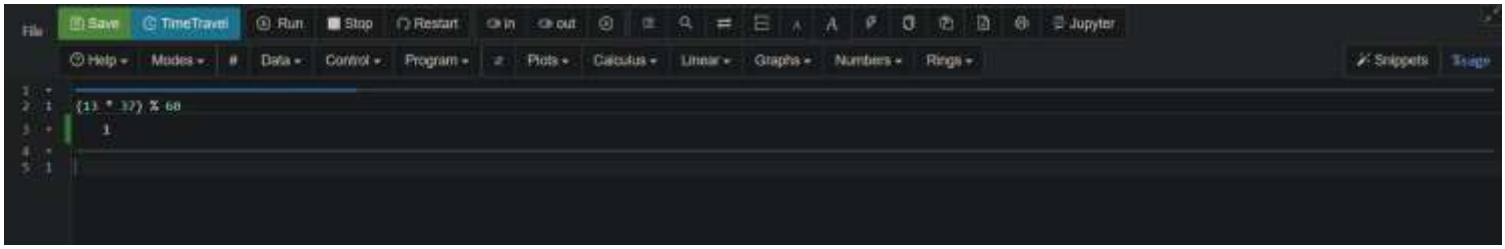
   E.g., let's say (**n = 77**) and (**e = 13**).

   The security of RSA relies on the difficulty of factoring **n** into its prime factors.
   In my example, (**n = 77**) can be factorized into (**p = 7**) and (**q = 11**).

   Once we have **p** and **q**, we can calculate Euler's totient function **ϕ(n) = (p - 1) (q - 1).**
   **ϕ(n) = (7 - 1) (11 - 1) = 60.**

   The private key (**d**) is the modular multiplicative inverse of **e modulo ϕ(n)**. It satisfies the equation (**ed mod ϕ(n)**).

   In my example, if (**e = 13**), we find (**d = 37**) because (**13 x 37 mod 60 = 1**).



We can factor **n** using various algorithms like pollard's rho algorithm, general number field sieve (GNFS), etc. Once we factor n, we can calculate the private key and decrypt message encrypted using public key.

However, as of now, factoring a large composite number efficiently is a computationally complex problem, and RSA encryption with sufficiently large key sizes remains secure.


External references for the above algorithms:


1. *A Beginner's Guide To The General Number Field Sieve*. (n.d.). UMD Computer Science| Michael Case. https://www.cs.umd.edu/~gasarch/TOPICS/factoring/NFSmadeeasy.pdf
2. GeeksforGeeks. (2023, June 21). *Pollard s Rho Algorithm for Prime Factorization*. GeeksforGeeks. https://www.geeksforgeeks.org/pollards-rho-algorithm-prime-factorization/
3. *Sagemath*. (2024, May 18). Cocalc. Retrieved May 18, 2024, from https://cocalc.com/projects/42c044fc-eb2d-47ec-b448-3b06ce3de3ba/files/Public%20Key%20Cryptography%20and%20Digital%20Signatures%20(Use%20RSA).sagews

2.

a. Let **Plaintext** be: 0101010101010101010101010101010101010101010101010101010101010101

i.

IP table

| *Initial Permutation* | | | | | | | |
|---|---|---|---|---|---|---|---|
| 58 | 50 | 42 | 34 | 26 | 18 | 10 | 02 |
| 60 | 52 | 44 | 36 | 28 | 20 | 12 | 04 |
| 62 | 54 | 46 | 38 | 30 | 22 | 14 | 06 |
| 64 | 56 | 48 | 40 | 32 | 24 | 16 | 08 |
| 57 | 49 | 41 | 33 | 25 | 17 | 09 | 01 |
| 59 | 51 | 43 | 35 | 27 | 19 | 11 | 03 |
| 61 | 53 | 45 | 37 | 29 | 21 | 13 | 05 |
| 63 | 55 | 47 | 39 | 31 | 23 | 15 | 07 |

To obtain the permuted output, we need to rearrange the bits of the plaintext according to the IP table. The table represents the new positions of the bits in the permuted output.

The first entry in the IP table is 58, which means the 58th bit of the plaintext (0) should be written as the first bit of the permuted output.

The second entry is 50, so the 50th bit of the plaintext (1) should be written as the second bit of the permuted output.

Continuing this process for all entries in the IP table, we get:
**Permuted Output:** 1111111111111111111111111111111100000000000000000000000000000000

ii. Assume Round Key: 0101010101010101010101010101010101010101010101010101010101010101

**Expansion Permutation:**
The 32-bit right half of the input (00000000000000000000000000000000) is expanded to 48 bits using the

Expansion Permutation: 000000000000000000000000000000000000000000000000

The expanded right half is XORed with the Round Key:
0101010101010101010101010101010101010101010101010101010101010101

S-box Substitution:

The 48-bit result from the XOR operation is divided into eight 6-bit blocks, and each block is substituted using the corresponding S-box.
Assuming the standard DES S-boxes, the result would be: 1001 0110 0100 0011 1100 1010 1011 0001

Concatenating these eight 4-bit values, the 32-bit output from the S-box Substitution would be:
10010110010000111100101010110001

Preparation for P-box Input:
The 32-bit output from the S-box Substitution is not directly used as input to the P-box. Instead, it needs to be divided into eight 4-bit blocks: 1001 0110 0100 0011 1100 1010 1011 0001

P-box Input:
These eight 4-bit blocks are then permuted according to the P-box permutation table:

P-box Permutation Table: 16 7 20 21 29 12 28 17 1 15 23 26 5 18 31 10 2 8 24 14 32 27 3 9 19 13 30 6 22 11 4 25

Applying the permutation table to the eight 4-bit blocks: 1101 0011 1110 0010 1000 0110 0000 1011

The resulting 32-bit value after applying the P-box permutation is: 11010011111000101000011000001011

This 32-bit value (11010011111000101000011000001011) is the output from the Permutation Function (P-box) step, which is then used in the subsequent XOR operation and swap steps.

XOR and SWAP

OLD LPT = 11111111111111111111111111111111
OLD RPT = 11010011111000101000011000001011

XOR this

New RPT = 00101100000111010111100111110100

Now the OLD RPT changes into the
NEW LPT = 00000000000000000000000000000000

Now this is the end of round one and the whole 64-bit output of this round is:

Output = 0000000000000000000000000000000010110000011101011110011111010 0

iii.

Assuming a Round Key, the cost of a Brute Force Attack can be calculated as follows:

DES uses a 56-bit key (64-bit key with 8 parity bits removed).
There are 2^56 ≈ 7.2 × 10^16 possible keys.

Given a computer that can run 10^13 instructions per second, the time required to try all possible keys would be:
**Time** = (Total Number of Keys) / (Instructions per Second)
= (7.2 × 10^16) / (10^13)
= 7.2 × 10^3 seconds
≈ 2 hours to Brute force this DES

b.

i.

**128-bit random number/ Plaintext/ hexadecimal:**

| | | | |
|---|---|---|---|
| E4 | 04 | 65 | 85 |
| 83 | 45 | 5D | 96 |
| 5C | 33 | 98 | B0 |
| F0 | 2D | AD | C5 |

**Sub bytes**

Plaintext gets compared with a table called s-box, to be substituted

S-box →

| | | | |
|---|---|---|---|
| 87 | F2 | 4D | 97 |
| EC | 6E | 4C | 90 |
| 4A | C3 | 46 | E7 |
| 8C | D8 | 95 | A6 |

**Shift Rows**

Rules of shifting rows:
Row 1 → no shift
Row 2 → 1 byte shift
Row 3 → 2 bytes shift
Row 4 → 3 bytes shift

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 87 | F2 | 4D | 97 | **1st row** | 87 | F2 | 4D | 97 |
| EC | 6E | 4C | 90 | **2nd row** | 6E | 4C | 90 | EC |
| 4A | C3 | 46 | E7 | **3rd row** | 46 | E7 | 4A | C3 |
| 8C | D8 | 95 | A6 | **4th row** | A6 | 8C | D8 | 95 |

**Mix Columns**

Each byte of a column is mapped into a new value that is a function of all four bytes in that column

| | | | |
|---|---|---|---|
| 02 | 03 | 01 | 01 |
| 01 | 02 | 03 | 01 |
| 01 | 01 | 02 | 03 |
| 03 | 01 | 01 | 02 |

X

| | | | |
|---|---|---|---|
| 87 | F2 | 4D | 97 |
| EC | 6E | 4C | 90 |
| 4A | C3 | 46 | E7 |
| 8C | D8 | 95 | A6 |

Predefined matrix State array

Let's Multiply the matrix

02 = 0000 0010 = x

87 = 1000 0111 = x^7 + x^2 + x + 1

02 x 87 = x (x^7 + x^2 + x + 1)

= x^8 + x^3 + x^2 + x

= Applying irreducible polynomial theorem, GF(2^3), x^8 = x^4 + x^3 + x + 1

= x^4 + x^3 + x + 1 + x^3 + x^2 + x

= x^4 + x^2 + 1

= 0001 0101

03 = 0000 0011 = x + 1

6E = 0110 1110 = x^6 + x^5 + x^3 + x^2 + x

03 x 6E = (x + 1) (x^6 + x^5 + x^3 + x^2 + x)

= x^7 + x^5 + x^4 + x

= 1011 0010

01 x 46 = 46

= 0100 0110

01 x A6 = A6

= 1010 0110

Now we need to XOR them all

02 x 87 = 0001 0101

03 x 6E = 1011 0010

01 x 46 = 1011 0010

01 x A6 = <u>1010 0110</u>

4 7

47 is the answer for the first box, similarly if we do it like that for all the boxes

New state array

| | | | |
|---|---|---|---|
| 47 | 40 | A3 | 4C |
| 37 | D4 | 70 | 9F |
| 94 | E4 | 3A | 42 |
| ED | A5 | A6 | BC |

**Add Round Key**

Add round key proceeds one column at a time

| | | | |
|---|---|---|---|
| 47 | 40 | A3 | 4C |
| 37 | D4 | 70 | 9F |
| 94 | E4 | 3A | 42 |
| ED | A5 | A6 | BC |

State

| | | | |
|---|---|---|---|
| AC | 19 | 28 | 57 |
| 77 | FA | D1 | 5C |
| 66 | DC | 29 | 00 |
| F3 | 21 | 41 | 6A |

Round key

47 XOR AC

47 = 0100 0111

AC = <u>1010 1100</u>

= 1110 1011

E B

37 XOR 77

37 = 0011 0111

77 = <u>0111 0111</u>

= 0100 0000

4 0

Similarly, do it for all the numbers

| | | | |
|---|---|---|---|
| EB | 59 | 8B | 1B |
| 4D | 2E | A1 | C3 |
| F2 | 38 | 13 | 42 |
| 1E | 84 | E7 | D6 |

= OUTPUT after running it through all four stages

ii.

AES-128 uses a 128-bit key, which means there are $2^{128} \approx 3.4 \times 10^{38}$ possible keys.

Given a computer that can run $10^3$ (1,000) instructions per second, the time required will be:

Time = (Total Number of Keys) / (Instructions per Second)

$= (3.4 \times 10^{38}) / (10^3)$

$= 3.4 \times 10^{35}$ seconds

$\approx 1.1 \times 10^{28}$ years

iii.

DES:

56-bit key

Brute Force Attack time: ≈ 2 hours (with $10^{13}$ instructions per second)

AES-128:

128-bit key

Brute Force Attack time: $\approx 1.1 \times 10^{28}$ years (with $10^3$ instructions per second)

The Brute Force Attack cost for AES-128 is significantly higher than that of DES due to the larger key size and the total time it takes for to break it. It is totally impractical to run a computer for years to break the AES encryption, but even though if we did, it would have to use up a lot of resources and which eventually costs more than breaking DE.

Even with a relatively slow computer ($10^3$ instructions per second), the time required to try all possible keys for AES-128 is practically infeasible.

So, AES encryption is much more efficient and secure than DES and costs more to break that.

3.

a.

A = (x^4 + x^3 + x^2 + x + 1)

B = (x^3 + x^2 +1)

A + B = (x^4 + x^3 + x^2 + x + 1) + (x^3 + x^2 +1)

= x^4 + x^3 + x^2 + x + 1 + x^3 + x^2 + 1

= x^4 + (1 + 1)x^3 + (1 + 1)x^2 + x + 1 + 1

= x^4 + x

A – B = (x^4 + x^3 + x^2 + x + 1) - (x^3 + x^2 +1)

= x^4 + x

A * B = (x^4 + x^3 + x^2 + x + 1) * (x^3 + x^2 +1)

= x^7 + x^6 + x^4 + x^6 + x^5 + x^3 + x^5 + x^4 + x^2 + x^4 + x^3 + x + x^3 + x^2 + 1

= x^7 + (1+1)x^6 + (1 + 1)x^5 + (1+1+1)x^4 + (1+1+1)x^3 + (1+1)x^2 + x + 1

= x^7 + x^4 + x^3 + x + 1

A / B = (x^4 + x^3 + x^2 + x + 1) / (x^3 + x^2 +1)

= x^3 + x^2 + 1 + [ (x^2 + 1) / x]

b. LFSR

i.

Seed: 0110

Shift all bits one position to the right

0110 -> 0011, the new leftmost bit is the XOR of the 1st and 4th bits of the previous state. Then repeat the steps before to generate more of the key stream.

| **Values** | **Key stream** |
|---|---|
| | |
| 0110 | |
| 0011 | 0 |
| 1001 | 1 |
| 0100 | 1 |
| 0010 | 0 |
| 0001 | 0 |
| 1000 | 1 |
| 1100 | 0 |
| 1110 | 0 |
| 1111 | 0 |
| 0111 | 1 |
| 1011 | 1 |
| 0101 | 1 |
| 1010 | 1 |
| 1101 | 0 |
| **0110** We are back to the starting position | 1 |

Key stream generated → 011001000111101

ii.

Length of key stream → (2^4 – 1) = 15 bits is the length of key stream is generated.

c.

Generated key → 011001000111101
Message → 11001010111010111001010101111

Let's use the key stream generated from LSFR to encrypt and decrypt a message of double the size of the key. We'll use XOR for encryption and decryption.

**Encryption**

To encrypt, we XOR the message with the key. We repeat the key to match the length of the message.

**Key →** 011001000111101011001000111101
**Message →** 11001010111010111001010101111

We XOR the both together

**Ciphertext →** 10101110100100010000010010010

**Decryption**

To decrypt, we XOR the ciphertext with the same key to get back the original message

**Ciphertext →** 10101110100100010000010010010
**Same key →** 011001000111101011001000111101

We XOR them

**Message →** 11001010111010111001010101111